

Logistic Regression

Hrachya Asatryan

Evaluation Metrics

Why do we need evaluation metrics?

Let's say we have trained a few models, that perform well on the train data. How can we compare said models? What sort of metrics can we use to choose the best model out of them, or to see if our model is even viable in the "real world"? For classification models, the most commonly used metrics are:

- Accuracy
- Precision
- Recall
- F1 Score
- Area Under Curve (AUC) and ROC Curve

Let's go over them and see their advantages and disadvantages.

Evaluation Metrics

	Actual positive	Actual negative
Predicted positive	True positive (TP): A spam email correctly classified as a spam email. These are the spam messages automatically sent to the spam folder.	False positive (FP): A not-spam email misclassified as spam. These are the legitimate emails that wind up in the spam folder.
Predicted negative	False negative (FN): A spam email misclassified as not-spam. These are spam emails that aren't caught by the spam filter and make their way into the inbox.	True negative (TN): A not-spam email correctly classified as not-spam. These are the legitimate emails that are sent directly to the inbox.

Evaluation Metrics

Accuracy:

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Advantages:

- Simple

Disadvantages:

- Can be misleading in cases of imbalanced datasets.
- Gives a False Positive sense of achieving high accuracy.

Evaluation Metrics

Precision:

Precision measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Advantages:

- Effective with Imbalanced Data
- Accuracy of Positive Predictions

Disadvantages:

- Ignores False Negatives, which can lead to overly conservative models that miss many actual positive cases.
- Focus on the Positive Class, as the metric doesn't account for the model's ability to find all relevant instances.

Evaluation Metrics

Recall (True Positive Rate):

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Advantages:

- In scenarios where catching all positive cases is important (like disease detection), recall is a key metric.

Disadvantages:

- Ignores False Positives, which can lead to overly liberal models that predict everything as positive.

Evaluation Metrics

- Does the patient have cancer? → cost of false negatives is high
Use Recall
- Is this wine good or not? → cost of false positives is high
Use Precision

Evaluation Metrics

F1 Score:

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number.

- Lower recall and higher precision gives us great accuracy but then it misses a large number of instances.
- A high F1 score means the model performs well on both metrics. Its range is [0,1].
- More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Evaluation Metrics

Area Under Curve (AUC) and ROC Curve:

First we need a few additional terms - **Specificity (True Negative Rate)**, which measures how many actual negative instances were correctly identified by the model. It answers the question: "Out of all the actual negative cases, how many did the model correctly identify as negative?"

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

And **FPR (False Positive Rate)** - It measures how many actual negative instances were incorrectly classified as positive.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Evaluation Metrics

Area Under Curve (AUC) and ROC Curve:

ROC Curve is the graphical representation of the **True Positive Rate (TPR)** vs the **False Positive Rate (FPR)** at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (**TPR**) and specificity (**1 - FPR**) across various thresholds.

Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

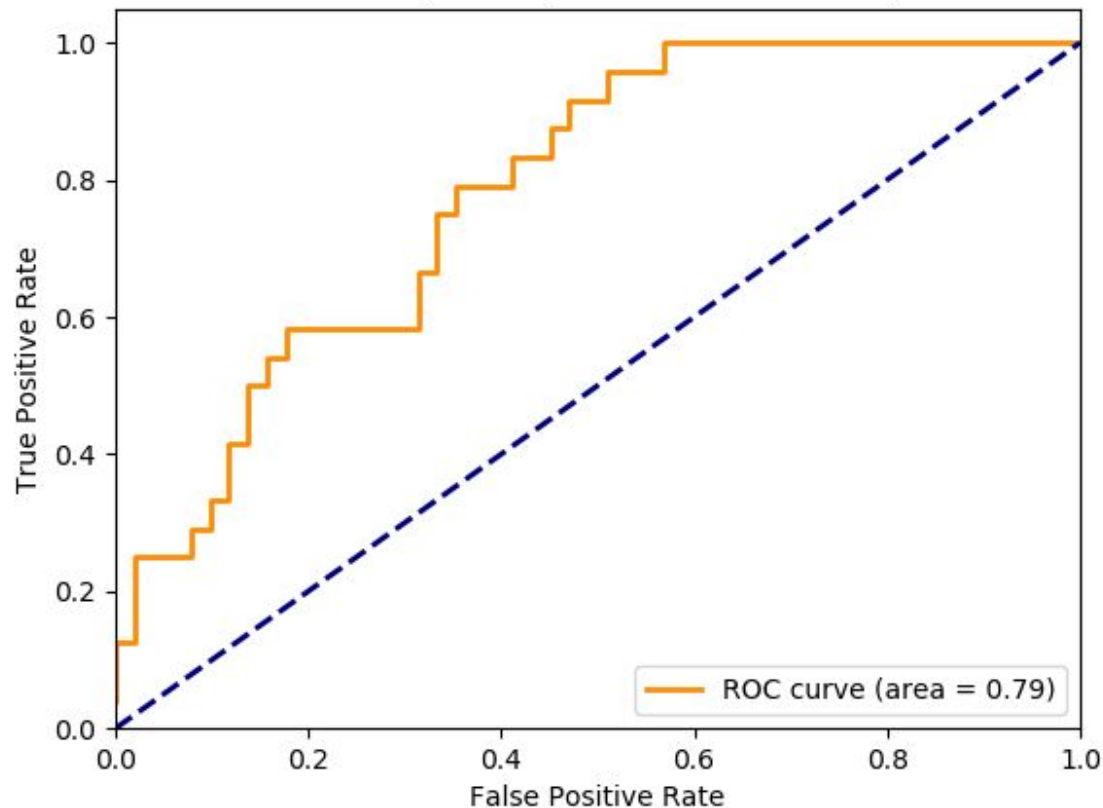
- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).

ROC curves are particularly useful when:

- Datasets are **imbalanced**
- Classification thresholds must be tuned

Evaluation Metrics

Receiver operating characteristic example



Model Evaluation

Training Error vs Test Error

As model complexity increases:

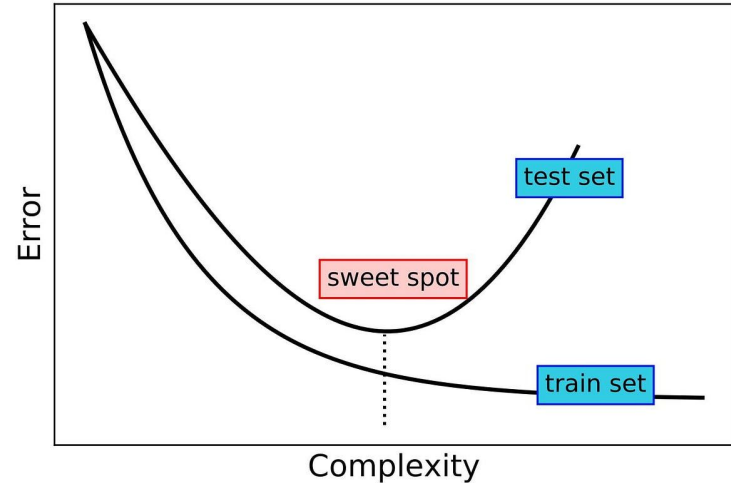
Training error ↓

Test error:

- Decreases initially
- Then increases due to **overfitting**

The optimal model lies at the **minimum test error**

Regularization helps us reach this point.



Classification

What do we have?

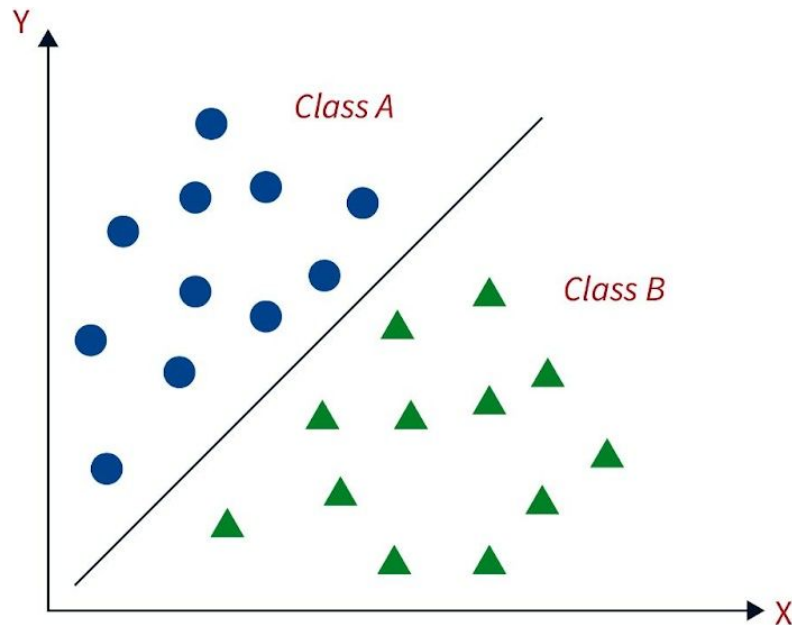
A training data set comprising **N** observations $\{x_n\}$, where $n = 1, \dots, N$, together with corresponding K discrete classes C_k where $k = 1, \dots, K$.

What is our goal?

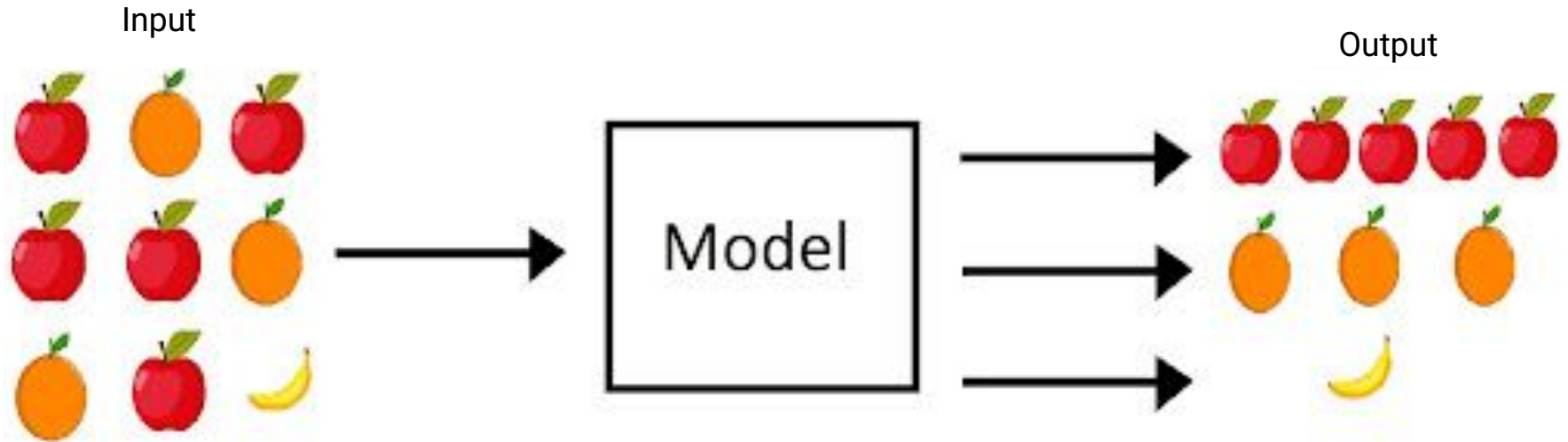
Take an input vector $\mathbf{x}$ and to assign it to one of K classes.

How do we do that?

We try to construct a function $y(\mathbf{x})$ that predicts the probability of class membership for new inputs $\mathbf{x}$, or more generally, we model the predictive distribution $p(t / \mathbf{x})$, where t is a discrete class label.



Classification



Classification

How do we classify the models?

The input space is divided into decision regions, whose boundaries are called decision boundaries or decision surfaces. We consider linear models for classification, meaning that the decision surfaces are linear functions of the input vector x , and hence are defined by $(D-1)$ -dimensional hyperplanes within the D -dimensional input space. Data sets whose classes can be separated exactly by such linear decision surfaces are said to be linearly separable.

For probabilistic models, the most convenient, in the case of two-class problems, is the binary representation in which there is a single target variable $t \in \{0, 1\}$ such that $t = 1$ represents class C_1 and $t = 0$ represents class C_2 . For $K > 2$ classes, it is convenient to use a 1-of- K coding scheme in which t is a vector of length K such that if the class is C_j , then all elements t_k of t are zero except element t_j , which takes the value 1. For instance, if we have $K = 5$ classes, then a pattern from class 2 would be given the target vector $\mathbf{t} = (0, 1, 0, 0, 0)^T$.

Classification

How do we classify the models?

We can tackle classification problems in multiple ways:

- The simplest way is to build a function that directly decides which class a new input x belongs to.
- A stronger method is to first estimate the probability that x belongs to each class $p(C_k | x)$, and then make the best decision based on these probabilities. Separating these steps (estimating probabilities first, then deciding) has many advantages.

There are two main ways to find these probabilities:

- Direct approach: Model the probabilities $p(C_k | x)$ directly and adjust the model using training data.
- Generative approach: Model how the data looks for each class $p(x | C_k)$ and how common each class is $p(C_k)$. Then, use Bayes' theorem to get $p(C_k | x)$.

Classification

From Linear to Nonlinear Output

In Linear Regression, the model prediction $\mathbf{y}(\mathbf{x}, \mathbf{w})$ was given by a linear function of the parameters $\mathbf{w}$. However for classification problems we need to predict discrete labels or posterior probabilities, thus we need to break the linearity using an **activation function** $f(\cdot)$, such that $y(x) = f(w^T x + w_0)$.

The decision surfaces correspond to $y(x) = \text{constant}$, so that $w^T x + w_0 = \text{constant}$ and hence the decision surfaces are linear functions of x , even if the function $f(\cdot)$ is nonlinear..

Classification

Two classes

The simplest form can be given as $y(x) = \text{sign}(w^T x + w_0)$, thus a negative y will result in x being assigned to C_2 and a positive means $x \rightarrow C_1$. Thus the corresponding boundary is $y(x) = 0$, which corresponds to a $(D - 1)$ -dimensional hyperplane within the D -dimensional input space.

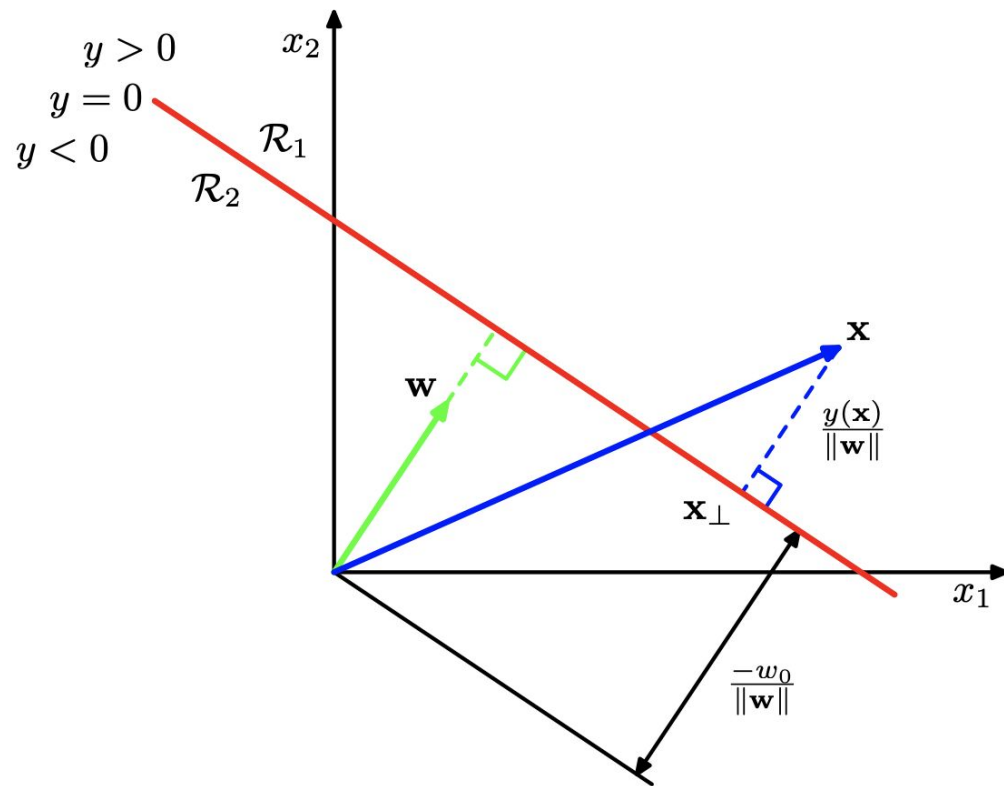
Consider two points x_A and x_B both of which lie on the decision surface. Because $y(x_A) = y(x_B) = 0$, we have $w^T(x_A - x_B) = 0$. Hence the vector w is orthogonal to every vector lying within the decision surface, and so w determines the orientation of the decision surface. Similarly, if x is a point on the decision surface, then $y(x) = 0$, and so the normal distance from the origin to the decision surface is given by:

$$\frac{w^T x}{\|w\|} = -\frac{w_0}{\|w\|}$$

And by breaking x down into an orthogonal and parallel components, we get, $r = \frac{y(x)}{\|w\|}$

Where r is the signed distance.

Classification



Classification

Multiple classes

Now consider the extension of linear discriminants to $K > 2$ classes. We might be tempted to build a K -class discriminant by combining a number of two-class discriminant functions. However, this leads to some serious difficulties.

We can treat this problem as

- One vs rest: we use $K-1$ classifiers each of which solves a two-class problem of separating points in a particular class C_k from points not in that class.
- One vs one: we introduce $K(K-1)/2$ binary discriminant functions, one for every possible pair of classes. Each point is then classified according to a majority vote amongst the discriminant functions.
- K class classification: we use a single K -class discriminant comprising K linear functions. Then assigning a point x to class C_k if $y_k(x) > y_j(x)$ for all $j \neq k$. The decision boundary between class C_k and class C_j is therefore given by $y_k(x) = y_j(x)$

Classification

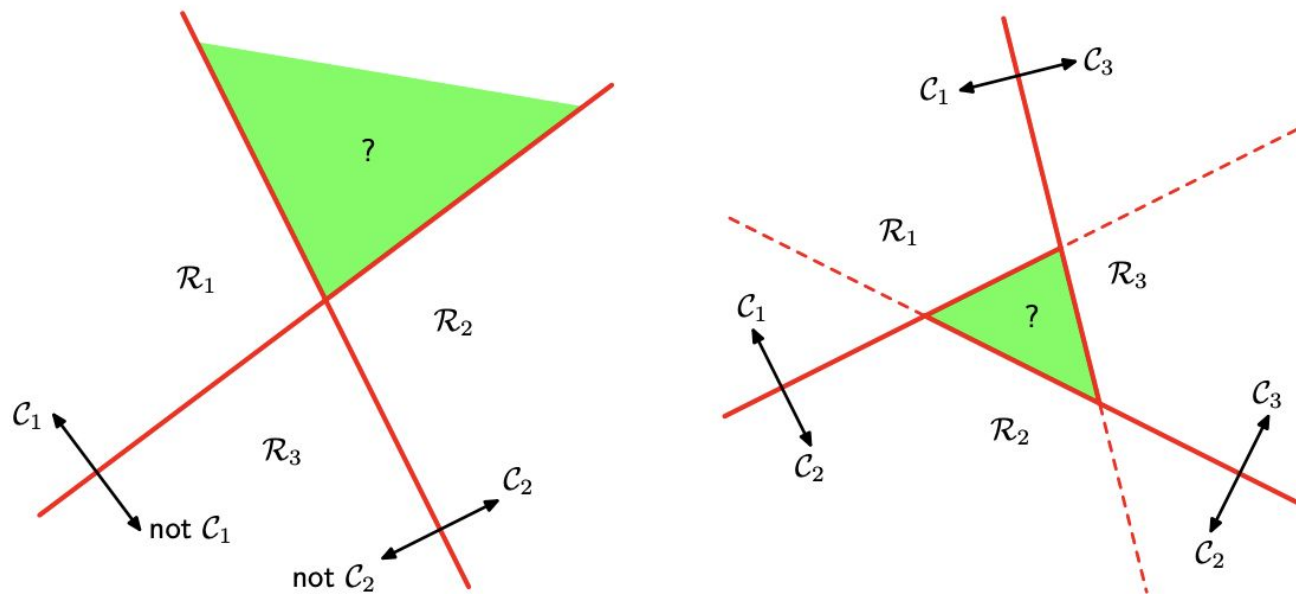


Figure 4.2 Attempting to construct a K class discriminant from a set of two class discriminants leads to ambiguous regions, shown in green. On the left is an example involving the use of two discriminants designed to distinguish points in class $\mathcal{C}_k$ from points not in class $\mathcal{C}_k$. On the right is an example involving three discriminant functions each of which is used to separate a pair of classes $\mathcal{C}_k$ and $\mathcal{C}_j$.

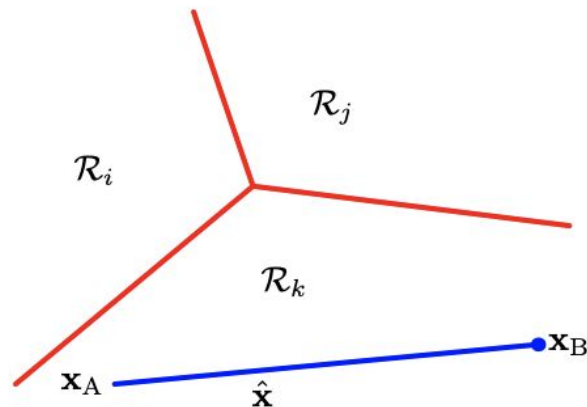
Classification

The decision regions of such a discriminant are always singly connected and convex. To see this, consider two points x_A and x_B both of which lie inside decision region R_k . Any point x that lies on the line connecting x_A and x_B can be expressed in the form: $x = \lambda x_A + (1 - \lambda)x_B$. For LDF-s it follows that:

$$y_k(\hat{x}) = \lambda y_k(x_A) + (1 - \lambda)y_k(x_B)$$

Because both x_A and x_B lie inside R_k , it follows that $y_k(x_A) > y_j(x_A)$, and $y_k(x_B) > y_j(x_B)$, for all $j \neq k$, and hence $y_k(x) > y_j(x)$, and so x also lies inside R_k . Thus R_k is singly connected and convex.

Illustration of the decision regions for a multiclass linear discriminant, with the decision boundaries shown in red. If two points x_A and x_B both lie inside the same decision region R_k , then any point $\hat{x}$ that lies on the line connecting these two points must also lie in R_k , and hence the decision region must be singly connected and convex.



Logistic Regression

From Regression to Classification

In Linear Regression, we predict **a real-valued output**

$$y = w^T x + b$$

But in classification, we want **a probability**

$$p(y=1|x)$$

Problem:

- Linear regression outputs values in $(-\infty, \infty)$
- Probabilities must lie in $[0, 1]$

Conclusion:

We need a function that maps real numbers $\rightarrow$ probabilities

Logistic Regression

Sigmoid Function

We use the sigmoid (logistic) function:

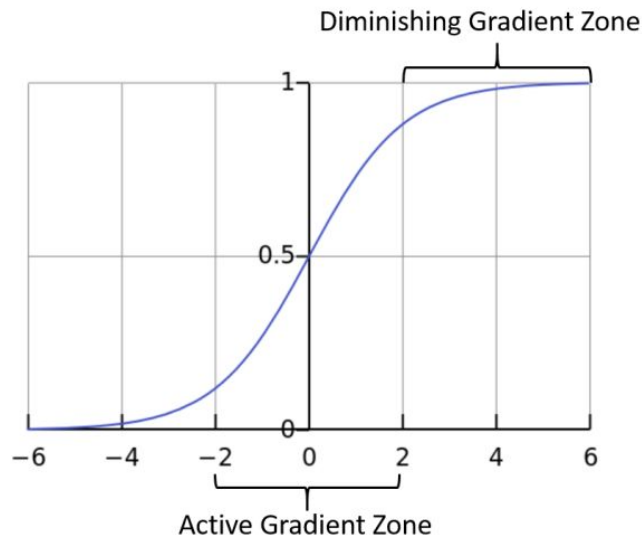
$$\sigma(x) = 1 / (1 + e^{-x})$$

Properties:

- Output is always between 0 and 1
- Smooth and differentiable
- Monotonically increasing

Model:

$$p(y=1|x) = \sigma(w^T x + b)$$



Logistic Regression

Main intuition:

Content:

$$p(y=1|x)=\sigma(w^T x+b)$$

- Output is interpreted as a probability
- If $p>0.5 \rightarrow$ predict class 1
- If $p<0.5 \rightarrow$ predict class 0

Decision Boundary:

$$w^T x+b=0$$

Important:

Even though we use a nonlinear function, the decision boundary is still linear

Logistic Regression

Modeling Probabilities

We assume:

- $y \in \{0, 1\}$
- Each output follows a Bernoulli distribution

$$p(y|x) = p^y (1-p)^{1-y}$$

where:

$$p = \sigma(w^T x + b)$$

Assumption:

- Data points are independent

Logistic Regression

Log Loss (Cross-Entropy)

We maximize likelihood $\rightarrow$ equivalently minimize negative log-likelihood:

$$\mathcal{L} = - \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Also called:

- Log Loss
- Cross-Entropy Loss

Key intuition:

- Strongly penalizes confident wrong predictions
- Encourages correct probabilities, not just correct labels

Logistic Regression

Optimization

Goal:

$$\min_w L(w)$$

Unlike linear regression:

- No closed-form solution

$$\nabla_w L(w) = \sum_{i=1}^N (\sigma(w^\top x_i) - y_i) x_i$$

Setting $\nabla_w L(w)=0$ yields a **nonlinear system which has no analytical solution for w .**

Gradient Descent

What is gradient descent?

Gradient Descent is

- An optimization technique/algorithm.
- Mostly used in supervised machine learning models and deep learning.
- Also called as first order optimization algorithm.
- One of the most used algorithms for optimization of parameters in ML models.

The meaning of Gradient Descent:

- The meaning of Gradient - first order derivative/ slope of a curve.
- The meaning of descent - movement to a lower point.
- The algorithm thus makes use of the gradient/slope to reach the minimum/ lowest point of a loss function.

The formula

While performing the algorithm of gradient descent, the machine iteratively calculates the next point it has to reach by using the gradient at the current position, and subtracting it from the parameter value by scaling it with a learning rate. The formula for the same looks like:

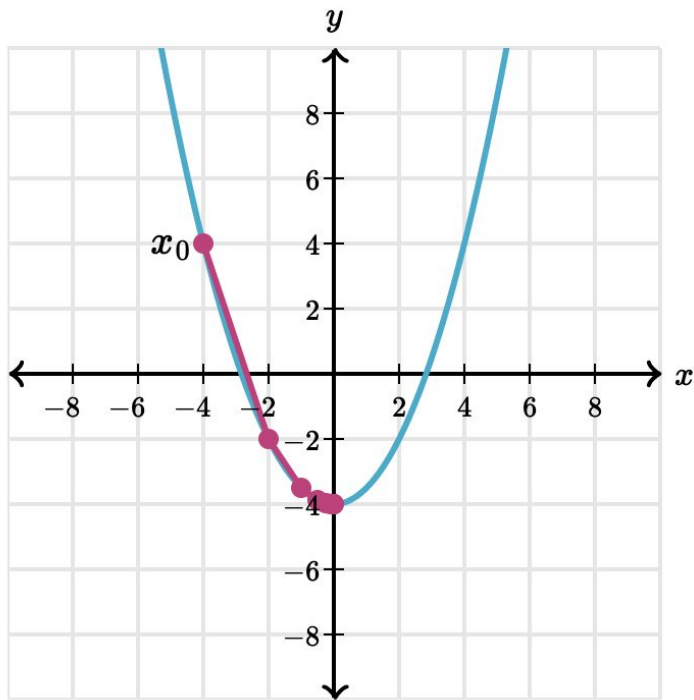
$$p_{n+1} = p_n - l \nabla f(p_n)$$

Where l is the learning rate, p_n is the parameter to be optimised, and $\nabla f(p_n)$ depicts the gradient of the expected loss function.

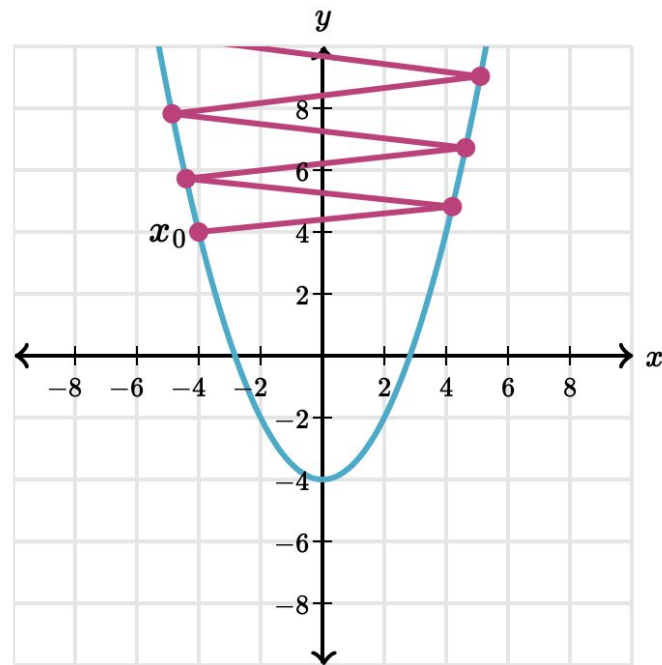
Learning rate

- The learning rate (LR) is an important parameter.
- It decides the rate at which the algorithm learns/ improves its unknown parameters
- The gradients alone are not enough to make sure we reach the best fit at a low computational time.
- The learning parameter η decides the step size of the learning/iteration.
- There can be three cases when we don't use the learning parameter:
 - The gradient is too small, and we reach the best fit point in too many iterations.
 - The gradient is too high, and we jump the best fit point to go farther away from it.
 - The gradient is just right, and we reach the best fit just fine.
- To make sure that the step size is just right, we use the learning parameter, which scales the change in the previous value of the parameter to get the new value

Learning rate

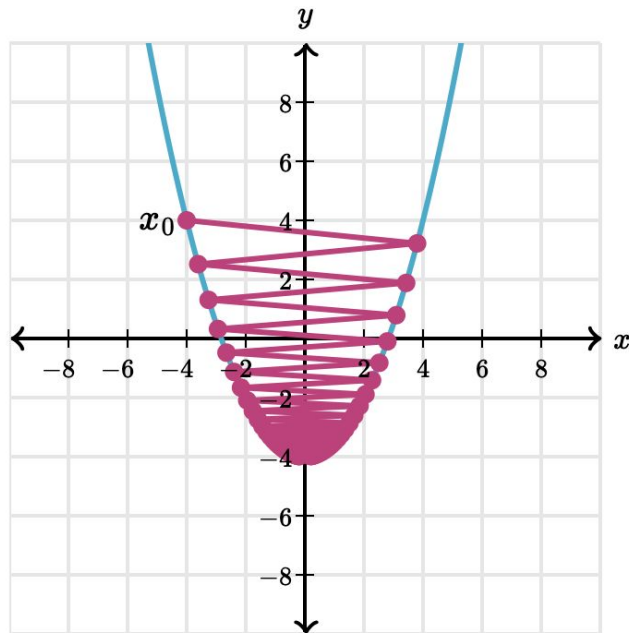


Good LR, converges quickly

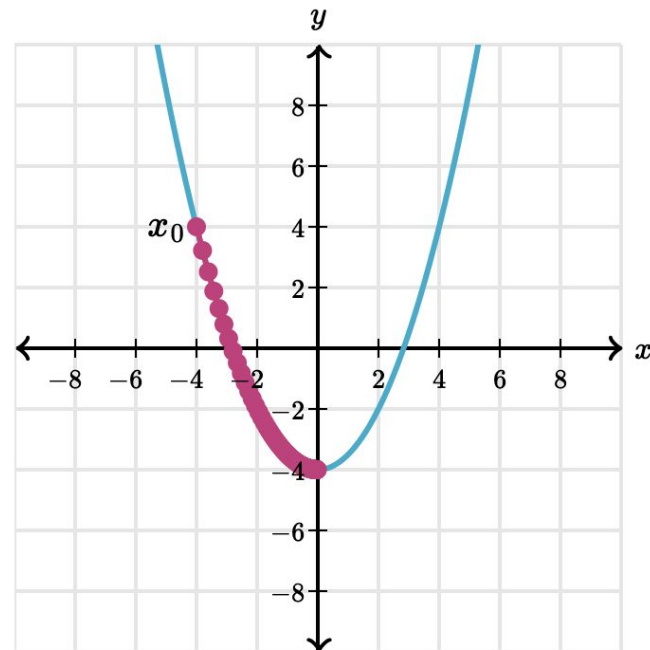


Large LR, diverges

Learning rate



Large LR, converges slowly



Small LR, converges slowly

Function requirements

- The gradient descent algorithm is not written for all types of functions.
- The function has to satisfy two conditions for Gradient Descent to be applicable on it:
 1. It has to be differentiable
 2. It has to be a convex function

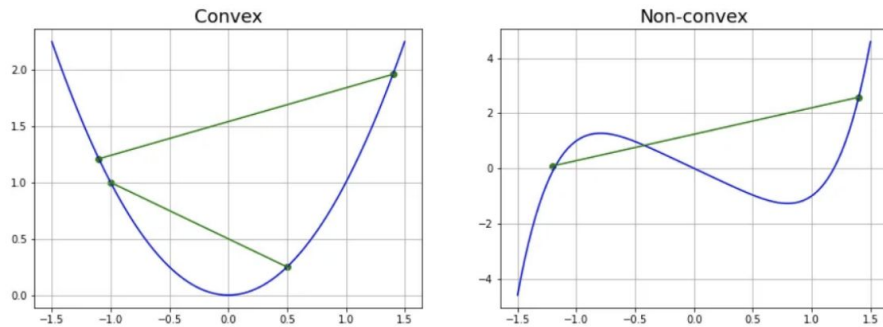


Figure. Example of convex and non-convex functions.[Kwiatkowski 2022]

Gradient descent for Logistic Regression

The loss (Cross-Entropy loss) is equal to

$$L(w) = - \sum_{i=1}^N [y_i \log \sigma(w^\top x_i) + (1 - y_i) \log(1 - \sigma(w^\top x_i))]$$

We need to find

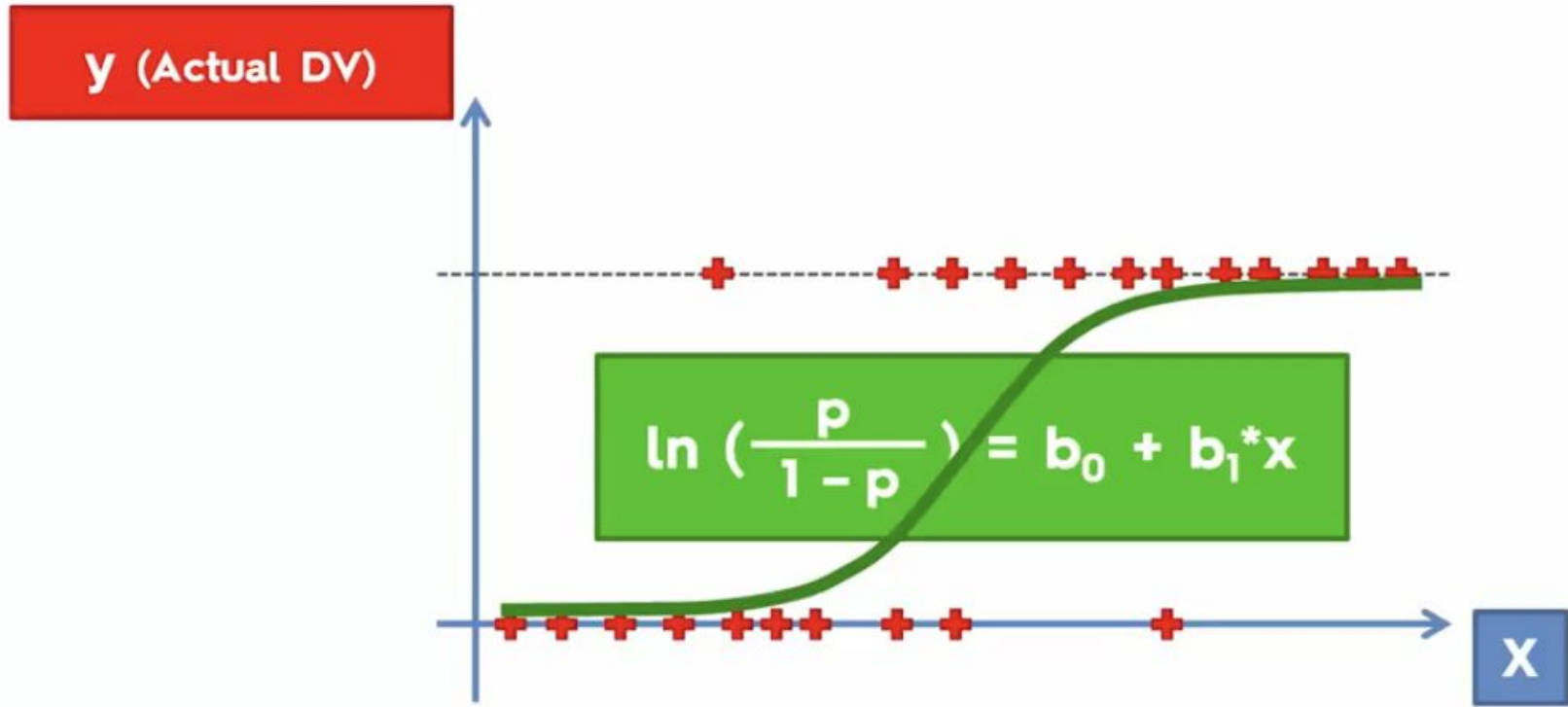
$$\min_w L(w)$$

Gradient exists, function is convex, but solving $\nabla L(w)=0$ analytically is impossible => **we use an optimization technique**. The gradient of the loss is

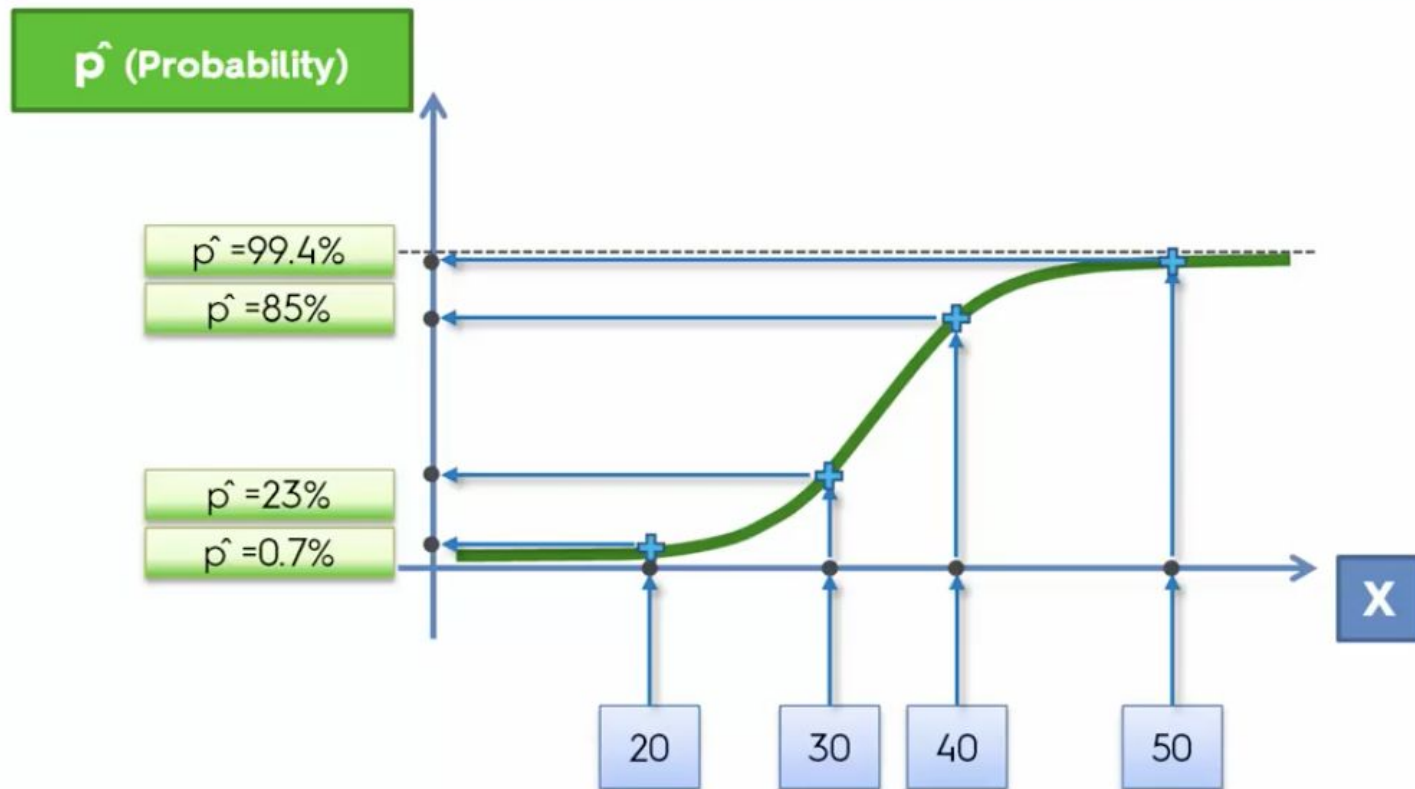
$$\nabla_w L(w) = \sum_{i=1}^N (\sigma(w^\top x_i) - y_i) x_i$$

We initialize $w^{(0)}$ and update iteratively: $w^{(t+1)} = w^{(t)} - \eta \sum_{i=1}^N (\sigma(w^{(t)\top} x_i) - y_i) x_i$

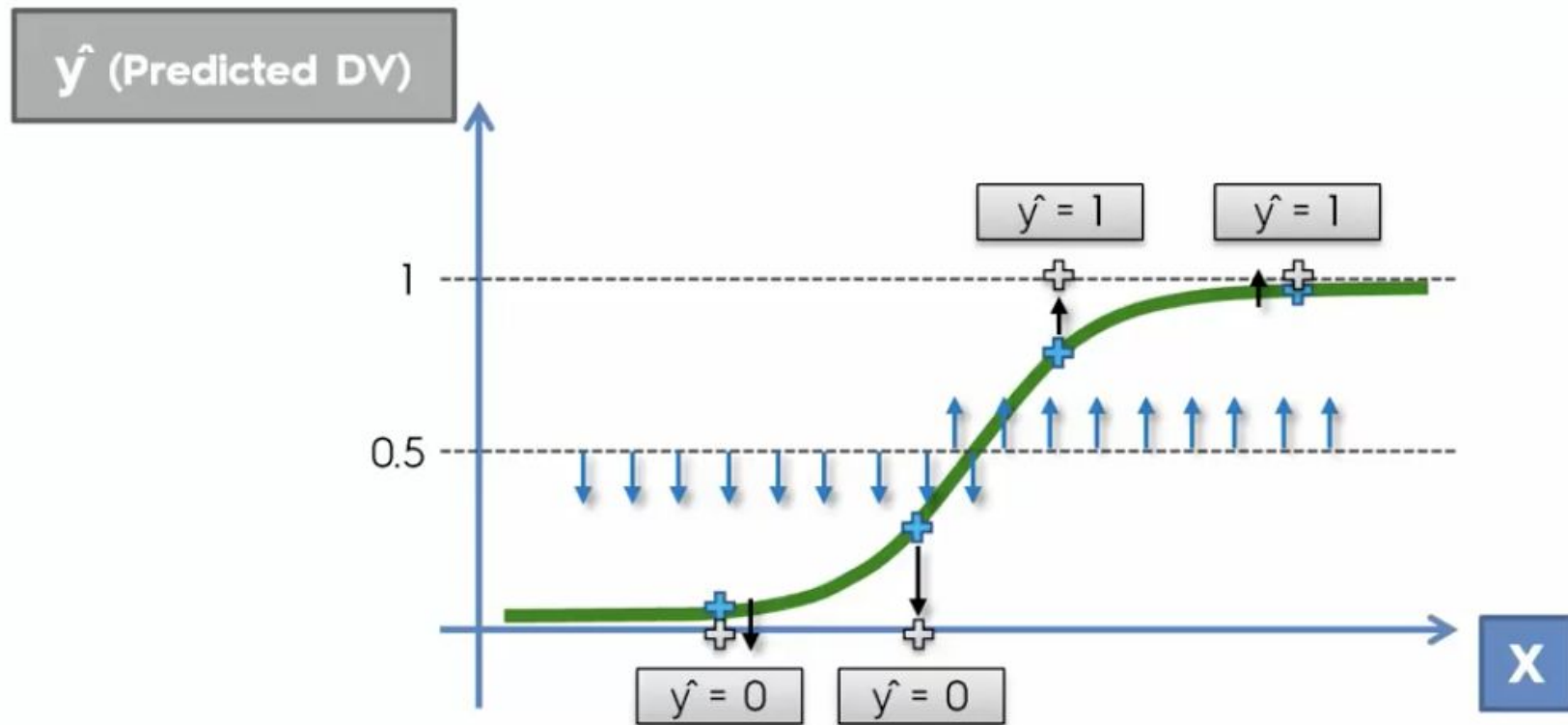
Logistic Regression



Logistic Regression



Logistic Regression



Logistic Regression

Multiclass Logistic Regression

So far:

- Binary classification ($y \in \{0,1\}$)

Now:

- Multiple classes ($y \in \{1,2,\dots,K\}$)

We model:

$$p(y=k | x)$$

Idea:

- Assign a score to each class
- Convert scores into probabilities

Logistic Regression

Softmax function

We use the softmax function:

$$p(y = k | x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

Properties:

- Outputs are all positive
- Sum of probabilities = 1
- Generalization of sigmoid to multiple classes

Interpretation:

- Higher score $\rightarrow$ higher probability

Logistic Regression

Training & Interpretation

We use cross-entropy loss (same idea as binary case):

$$\mathcal{L} = - \sum_i \sum_k y_{ik} \log p_{ik}$$

where:

- $y_{ik}=1$ if sample i belongs to class k , else 0

Prediction:

$$\text{class} = \operatorname{argmax}_k p(y=k|x)$$

Same as logistic regression, but extended to multiple classes

Least Squares for Classification

- Consider a general classification problem with K classes, with a 1-of- K binary coding scheme for the target vector $\mathbf{t}$.
- Each class C_k is described by its own linear model so that

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

- Thus, if we put these together, we obtain:

$$\mathbf{y}(\mathbf{x}) = \widetilde{\mathbf{W}}^T \widetilde{\mathbf{x}}$$

Least Squares for Classification

- Consider training data set $\{\mathbf{x}_n, t_n\}$
- We compute the coefficient matrix by minimizing a sum-of-squares error function

$$\frac{1}{2} \text{Tr} \left\{ (\tilde{\mathbf{X}} \tilde{\mathbf{W}} - \mathbf{T})^T (\tilde{\mathbf{X}} \tilde{\mathbf{W}} - \mathbf{T}) \right\}$$

- Setting the derivative with respect to the weight matrix, to zero we then obtain

$$\tilde{\mathbf{W}} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{T} = \tilde{\mathbf{X}}^\dagger \mathbf{T}$$

- The discriminant function would be

$$\mathbf{y}(\mathbf{x}) = \tilde{\mathbf{W}}^T \tilde{\mathbf{x}}$$

Least Squares for Classification

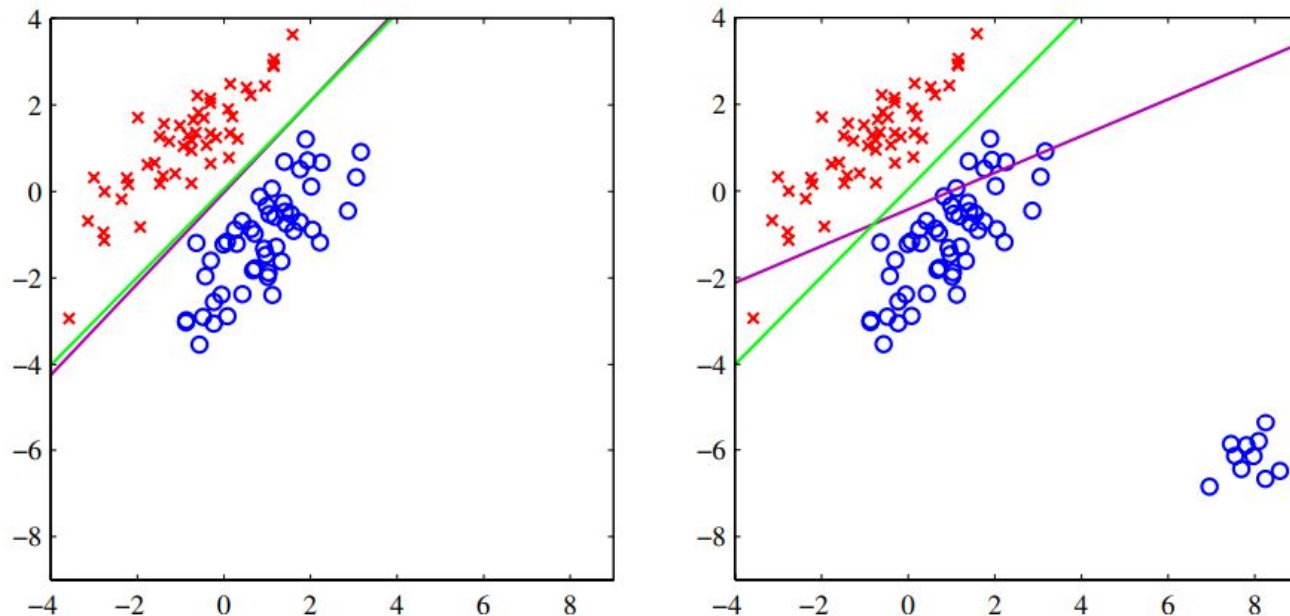


Figure 4.4 The left plot shows data from two classes, denoted by red crosses and blue circles, together with the decision boundary found by least squares (magenta curve) and also by the logistic regression model (green curve), which is discussed later in Section 4.3.2. The right-hand plot shows the corresponding results obtained when extra data points are added at the bottom left of the diagram, showing that least squares is highly sensitive to outliers, unlike logistic regression.

Least Squares for Classification

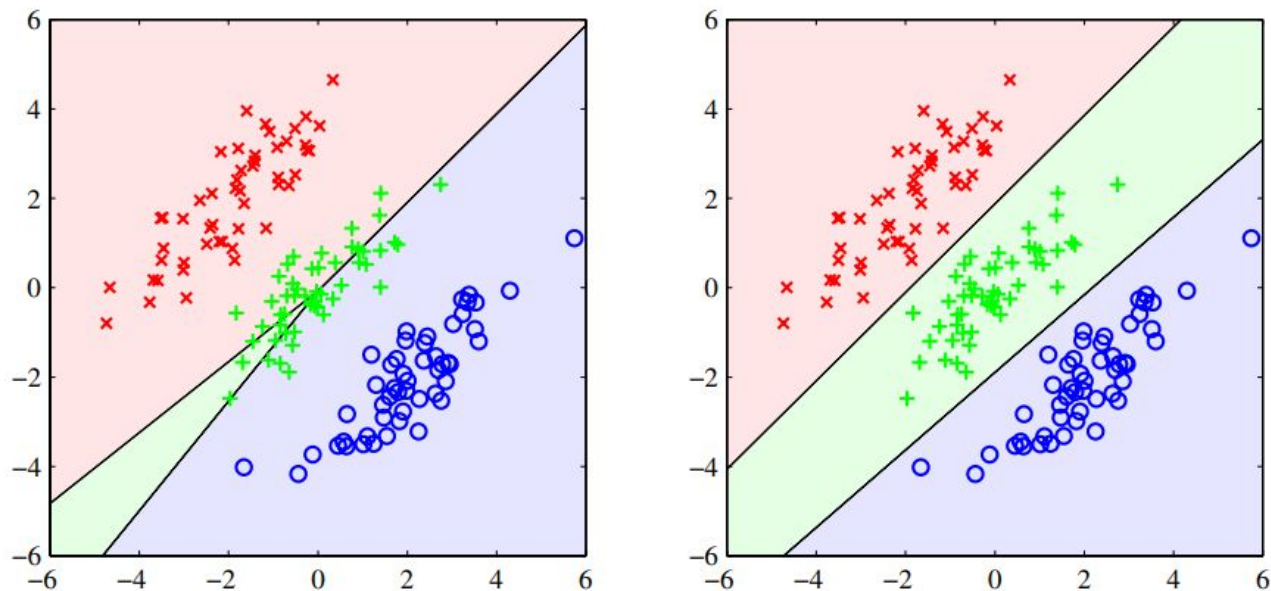


Figure 4.5 Example of a synthetic data set comprising three classes, with training data points denoted in red ($\times$), green ($+$), and blue ($\circ$). Lines denote the decision boundaries, and the background colours denote the respective classes of the decision regions. On the left is the result of using a least-squares discriminant. We see that the region of input space assigned to the green class is too small and so most of the points from this class are misclassified. On the right is the result of using logistic regressions as described in Section 4.3.2 showing correct classification of the training data.

Fisher's Linear Discriminant

- We view the linear classification model in terms of dimensionality reduction.
- Assume two classes and project $y = \mathbf{w}^T \mathbf{x}$.
- If we place a threshold on y and classify $y > w_0$ as class C_1 , and otherwise class C_2 , then we obtain our standard linear classifier discussed in the previous section.
- The projection onto one dimension leads to a considerable loss of information.
- Classes that are well separated in the original D -dimensional space may become strongly overlapping in one dimension.
- However, by adjusting the components of the weight vector $\mathbf{w}$, we can select a projection that maximizes the class separation.

Fisher's Linear Discriminant

- To begin with, consider a two-class problem in which there are N_1 points of class C_1 and N_2 points of class C_2 .

$$\mathbf{m}_1 = \frac{1}{N_1} \sum_{n \in C_1} \mathbf{x}_n, \quad \mathbf{m}_2 = \frac{1}{N_2} \sum_{n \in C_2} \mathbf{x}_n.$$

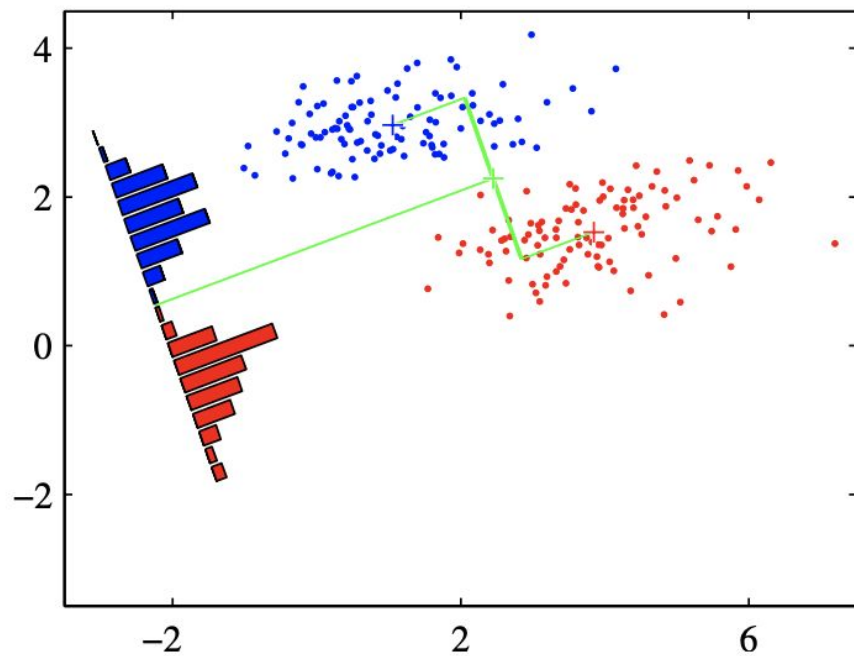
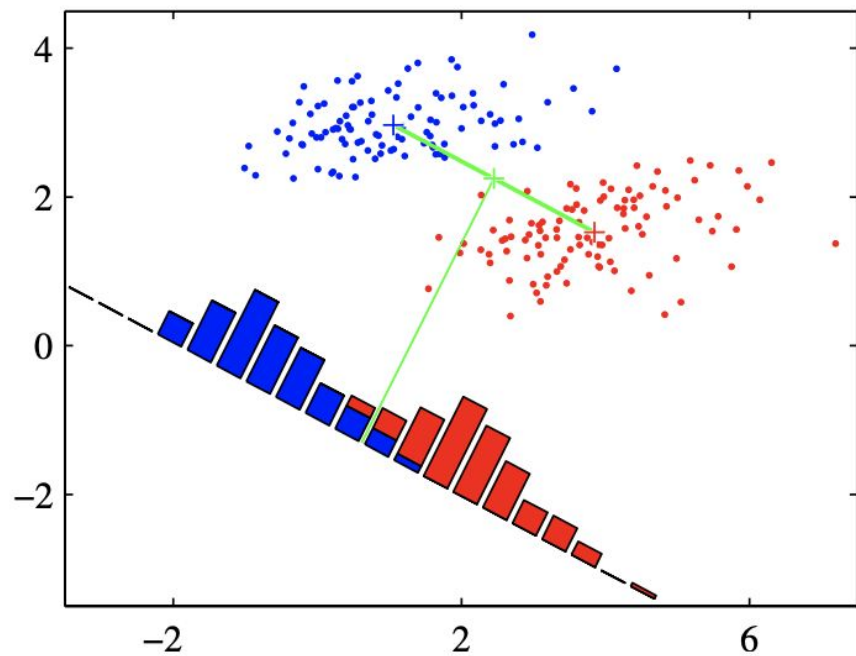
- The simplest measure of the separation of the classes, when projected, is the separation of the projected class means. This suggests that we might choose $\mathbf{w}$ so as to maximize

$$m_2 - m_1 = \mathbf{w}^T (\mathbf{m}_2 - \mathbf{m}_1) \quad \text{where} \quad m_k = \mathbf{w}^T \mathbf{m}_k$$

Fisher's Linear Discriminant

- However, this expression can be made arbitrarily large simply by increasing the magnitude of w .
- To solve this problem, we could constrain w to have unit length, so that $\|w\|_2 = 1$. By using Lagrange multipliers we obtain $w \sim (m_2 - m_1)$.
- There is still a problem with this approach, however, see the figure in next slide (This shows two classes that are well separated in the original two dimensional space but that have considerable overlap when projected onto the line joining their means.
- This difficulty arises from the strongly non-diagonal covariances of the class distributions.

Fisher's Linear Discriminant



Fisher's Linear Discriminant

- The idea proposed by Fisher is to maximize a function that will give a large separation between the projected class means while also giving a small variance within each class, thereby minimizing the class overlap.
- The within-class variance of the transformed data from class C_k is given by

$$s_k^2 = \sum_{n \in C_k} (y_n - m_k)^2$$

- The Fisher criterion is defined to be the ratio of the between-class variance to the within-class variance and is given by

$$J(\mathbf{w}) = \frac{(m_2 - m_1)^2}{s_1^2 + s_2^2}.$$

- We next rewrite the Fisher criterion

Fisher's Linear Discriminant

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

where $\mathbf{S}_B$ is the *between-class* covariance matrix and is given by

$$\mathbf{S}_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^T$$

and $\mathbf{S}_W$ is the total *within-class* covariance matrix, given by

$$\mathbf{S}_W = \sum_{n \in \mathcal{C}_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^T + \sum_{n \in \mathcal{C}_2} (\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^T.$$

Fisher's Linear Discriminant

- Differentiating $J(\mathbf{w})$ with respect to $\mathbf{w}$, we find that $J(\mathbf{w})$ is maximized when

$$(\mathbf{w}^T \mathbf{S}_B \mathbf{w}) \mathbf{S}_W \mathbf{w} = (\mathbf{w}^T \mathbf{S}_W \mathbf{w}) \mathbf{S}_B \mathbf{w}.$$

- We see that $\mathbf{S}_B \mathbf{w}$ is always in the direction of $(\mathbf{m}_2 - \mathbf{m}_1)$.
- Furthermore, we do not care about the magnitude of $\mathbf{w}$, only its direction, and so we can drop the scalar factors $(\mathbf{w}^T \mathbf{S}_B \mathbf{w})$ and $(\mathbf{w}^T \mathbf{S}_W \mathbf{w})$.
- Multiplying both sides of by $\mathbf{S}_W^{-1}$ we then obtain

$$\mathbf{w} \propto \mathbf{S}_W^{-1} (\mathbf{m}_2 - \mathbf{m}_1).$$